SIMATS ENGINEERING

ASSESSMENT TOOL 2: Error Analysis Task

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Register Number	192572101
Name	M.Kyathi Sai Priya

COURSE OUTCOME COVERED

CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 25**

Question Count: 5

Code of Conduct

I M. Kyathi Sai Priya (192572101) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: M.Kyathi Sai Priya

Assessment Tasks

Error Analysis Task

1. A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.
2. An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.
3. A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.
4. A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.
5. A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Error Analysis Task Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Identification of Error	30%	Accurately identifies the root technical issue	Identifies most of the issue	Partially identifies issue	Fails to identify issue
Cause Analysis	20%	Explains root cause with strong technical reasoning	Reasonable cause analysis	Basic cause analysis	Weak analysis
Corrective Action	25%	Proposes effective and feasible corrections	Reasonable corrections	Basic corrections	Ineffective corrections
Use of Hadoop / Integration Concepts	15%	Applies relevant concepts appropriately	Mostly appropriate application	Partial application	Incorrect application
Clarity	10%	Clear and direct explanation	Generally clear	Somewhat unclear	Poorly presented

1. Question

A Hadoop job repeatedly fails because input splits are uneven and reducers remain idle. Identify the likely design error and propose corrective actions.

Answer

The problem indicates an **improper input-split and partitioning design**. If input splits are highly unequal, some Mapper tasks receive much more data than others. This causes **data skew** and creates a bottleneck. Reducers may remain idle because they are waiting for the slowest Mapper tasks or because the partitioning strategy sends little data to some reducers.

Likely Design Errors

1. Input files have highly different sizes.
2. HDFS blocks and input splits are not properly balanced.
3. Very large files may be combined with small files inefficiently.
4. The Mapper output keys may be highly skewed.
5. The number of reducers may be too high compared with the amount of intermediate data.
6. A poor partitioner may send most records to only a few reducers.

7. Data locality may be poor, increasing network transfer.

Corrective Actions

- Use appropriate HDFS block sizes.
- Split large input files into reasonably balanced chunks.
- Use a **custom InputFormat** if the default splitting is unsuitable.
- Use a **Combiner** to reduce intermediate data.
- Analyze key distribution to identify data skew.
- Use a suitable partitioner to distribute keys more evenly.
- Configure an appropriate number of reducers.
- Enable compression of intermediate data.
- Use speculative execution for slow tasks when appropriate.

Example

If Mapper outputs are:

Reducer 1 → 80% of data

Reducer 2 → 10% of data

Reducer 3 → 5% of data

Reducer 4 → 5% of data

Reducer 1 becomes a bottleneck while the other reducers become idle.

A better partitioning strategy distributes the records more evenly.

2. Question

An HDFS deployment shows frequent data unavailability after a single node failure. Analyze the probable replication or NameNode design error.

Answer

Frequent data unavailability after a single DataNode failure indicates that the HDFS cluster may have **insufficient replication or poor high-availability configuration**.

Likely Problems

1. Low Replication Factor

If the replication factor is set to **1**, a block exists on only one DataNode.

Block A → DataNode 1

If DataNode 1 fails, Block A becomes unavailable.

A replication factor of **3** is generally preferable for important data:

Block A → DataNode 1

→ DataNode 2

→ DataNode 3

2. Poor Replica Placement

If all replicas are placed on the same rack, a rack failure can affect all copies.

Rack awareness should distribute replicas across racks.

3. NameNode Single Point of Failure

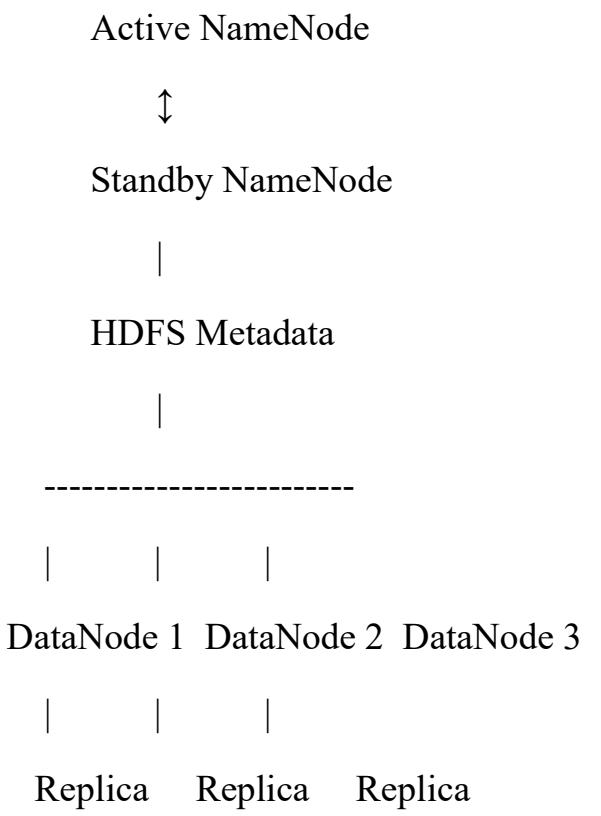
If there is only one NameNode, its failure can make the entire HDFS namespace unavailable even when DataNodes are healthy.

Corrective Actions

- Set appropriate replication, commonly up to 3 for critical data.
- Use **rack-aware replica placement**.
- Configure **Active and Standby NameNodes**.
- Use JournalNodes for NameNode shared metadata.
- Configure automatic failover.
- Monitor DataNode heartbeats.
- Run HDFS fsck and replication checks.

- Re-replicate under-replicated blocks automatically.

Improved Architecture



3. Question

A data ingestion workflow using ETL and Apache NiFi creates duplicate records in the analytics store. Identify the likely causes and recommend fixes.

Answer

Duplicate records usually occur when the ingestion pipeline does not properly track already-processed data or when retry mechanisms insert the same record more than once.

Likely Causes

1. Repeated File Processing

NiFi may process the same input file multiple times if file tracking or state management is incorrectly configured.

2. Lack of Unique Identifier

If records do not have a unique transaction ID, the analytics store cannot easily identify duplicates.

3. Retry Without Idempotency

If a transfer fails after the destination receives the data, NiFi may retry the same record, creating a duplicate.

4. ETL Reprocessing

An ETL job may read the same source data repeatedly without maintaining an incremental-load timestamp or checkpoint.

5. Incorrect NiFi Flow

Incorrect use of processors such as GetFile, FetchFile, or MergeContent can result in repeated processing.

Corrective Actions

- Assign a unique ID to every record.
- Use NiFi state management to track processed data.
- Use checkpoints or watermarks for incremental ingestion.
- Configure proper retry and failure relationships.
- Use idempotent writes.
- Apply database **primary keys or unique constraints**.
- Use record-level deduplication.
- Generate hashes for duplicate detection.
- Separate successful and failed records.

Example

Suppose:

Transaction ID = T1001

Amount = ₹500

If T1001 is already stored, another incoming T1001 should be rejected or updated instead of inserted again.

Improved Pipeline

Source

↓

NiFi

↓

Validation

↓

Unique ID / Hash Check

↓

Deduplication

↓

ETL Transformation

↓

Analytics Store

4. Question

A YARN cluster suffers from resource starvation when multiple users submit jobs together. Analyze the scheduling error and suggest a better resource management approach.

Answer

Resource starvation occurs when one user or application consumes most of the available cluster resources, leaving insufficient CPU and memory for other users.

Likely Scheduling Errors

1. No proper queue configuration.
2. A single queue is used for all users.
3. No minimum resource allocation is defined.
4. No maximum resource limit exists.
5. One large application can consume most cluster resources.

6. User/application limits are not configured.
7. Scheduler priorities are poorly defined.
8. Fair sharing is not enabled or properly configured.

Corrective Actions

1. Use Capacity Scheduler

Create separate queues:

Root

├── ETL

├── Reporting

├── MachineLearning

└── General

Each queue receives a minimum capacity.

2. Configure Maximum Capacity

A single queue should not consume the entire cluster.

For example:

ETL → 40%

Reporting → 20%

ML → 30%

General → 10%

Unused resources can be shared dynamically.

3. User Limits

Set limits on the resources that each user or application can consume.

4. Priority

Critical jobs can receive higher priority, but lower-priority jobs should not be permanently starved.

5. Fair Scheduling

Resources should be distributed fairly among users and queues.

6. Preemption

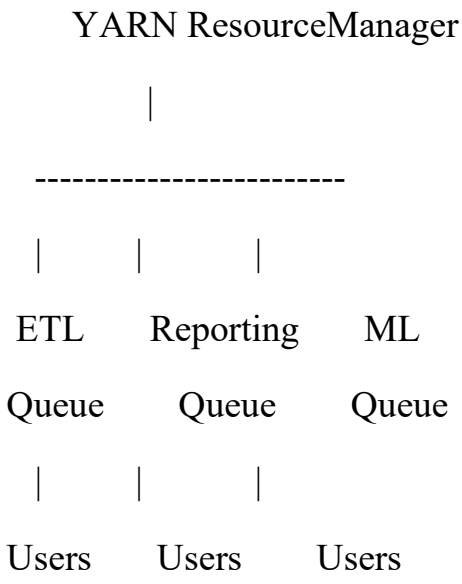
If one queue is using more than its allowed share, resources can be reclaimed within configured policies.

7. Monitoring

Monitor:

- CPU utilization
- Memory utilization
- Queue utilization
- Running applications
- Pending applications

Improved Architecture



5. Question

A backup strategy for distributed storage restores outdated data after recovery. Identify the probable design flaw in the replication or backup approach.

Answer

Restoring outdated data indicates that the backup system is not properly synchronized with the current data. The problem may be caused by **stale backups, incorrect backup scheduling, replication lag, or confusion between replication and backup**.

Likely Design Flaws

1. Outdated Backup

The backup may run only once a day. If the system fails before the next backup, recent data is lost.

2. Replication Mistaken for Backup

HDFS replication protects against node failure but does not provide historical recovery.

For example:

Current Data → Replica 1

→ Replica 2

→ Replica 3

If data is accidentally deleted, replication can propagate the deletion. Replication alone is therefore not a complete backup strategy.

3. Incorrect Backup Scheduling

Backups may not run frequently enough.

4. No Versioning

Without backup versions or snapshots, only the latest copy may be available.

5. Backup Verification Missing

A backup may appear successful but contain incomplete or stale data.

Corrective Actions

- Use regular incremental backups.
- Perform periodic full backups.
- Maintain backup versions.

- Use HDFS snapshots where appropriate.
- Maintain timestamps/checkpoints.
- Monitor backup completion.
- Verify backup integrity.
- Keep backups in independent storage.
- Test restoration periodically.
- Maintain a defined **RPO (Recovery Point Objective)** and **RTO (Recovery Time Objective)**.

Example

If the business requires an RPO of 15 minutes:

10:00 → Backup

10:15 → Backup

10:30 → Backup

10:45 → Backup

If a failure occurs at 10:50, the maximum expected data loss is approximately 5 minutes, assuming the backup completed successfully.

Improved Backup Architecture

Primary HDFS

↓

Snapshots / Incremental Backup

↓

Secondary Storage

↓

Versioned Backup

↓

Recovery / Restore